

Single-Port RAM

RTL Design and Functional Verification

Project Report

submitted by

Shri Hari Rao Emp ID: 6933

Miraфра Software Technologies Pvt. Ltd

July 2026

Table of Contents

1. Project Introduction	3
2. Objectives	4
3. Design Specification	5
Block Diagram	5
Pin-Out Diagram / Pin Description	6
4. Memory Operation Timing	7
Write Operation Timing	7
Read Operation Timing	8
5. Testbench Architecture	9
Testbench Block Diagram	9
Component Descriptions	9
6. Quality Assessment — Code Coverage	11
7. Results	12
Simulation Results	12
Documented Bug — Reset / Reference-Model Mismatch	12
8. Conclusion	13
9. Future Work	14

1. Project Introduction

What is this project

This project presents the RTL design and functional verification of a single-port Random Access Memory (RAM), implemented in synthesizable Verilog HDL and verified with a SystemVerilog class-based testbench. The design is developed as part of the Mirafra Technologies training program, following industry-standard RTL design and verification practices.

What is a single-port RAM

A single-port RAM is a memory block that exposes a single address/data interface shared between read and write operations. Because only one port is available, the design does not support simultaneous read and write access to the memory in the same cycle - each cycle can perform at most one operation.

Design summary

The RAM operates at 50 MHz, is 8 bits wide and 32 locations deep, and is controlled by separate `write_enb` and `read_enb` signals along with a 5-bit address bus. Reset is used to bring the design to a known idle state before normal operation begins.

Verification approach

The RAM is verified using a SystemVerilog class-based testbench following the classic generator-driver-monitor-scoreboard architecture, with an independent behavioural reference model used as the golden model for functional comparison. The testbench automatically compares DUV outputs against the reference model for every read operation and reports MATCH or MISMATCH counts, along with functional coverage of the input and output spaces.

2. Objectives

- Study and understand the single-port RAM architecture and pin-level description as specified in the RAM Design Specification document.
- Implement a synthesizable RTL design in Verilog HDL covering write and read operations with correct handling of clock and reset.
- Design and implement a behavioural reference model (ram_reference_model) that serves as the golden model for functional comparison during verification.
- Develop a structured, class-based SystemVerilog testbench (transaction, generator, driver, monitor, reference model, scoreboard, environment) that automatically drives inputs, compares outputs, and reports results.
- Verify write operation, read operation, read-after-write correctness, reset behaviour, and the no-simultaneous-read-write constraint.
- Run a consolidated regression test covering base random, write-only, and read-only scenarios in a single simulation session.
- Achieve maximum functional and code coverage including statement, branch, toggle, and functional (covergroup) coverage through comprehensive test case selection.

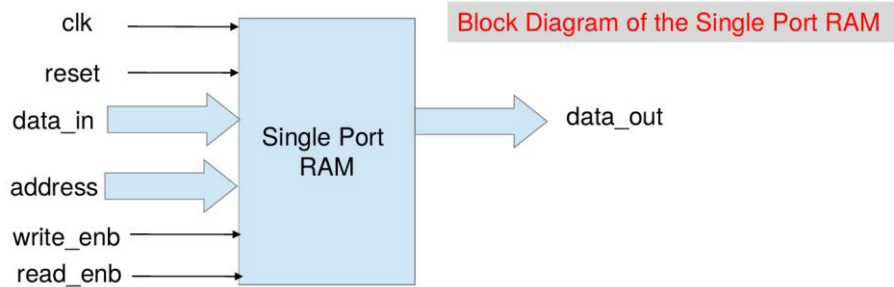
3. Design Specification

This is a single-port RAM. The operating frequency for the RAM is 50 MHz. The RAM is 8 bits wide and 32 locations deep. When reset is asserted, the RAM is in idle state - all inputs become zero except data input, which becomes high-impedance (Z), and the data output also becomes Z. The RAM supports both read and write operations, and it does not support concurrent read and write operations.

Block Diagram

1. Design Overview, Functional Description and Block Diagram

- This is a single port RAM
- The operating frequency for the RAM is 50MHz.
- This RAM is 8 bits wide and 32 locations deep.
- When the reset is asserted the RAM is in idle state, i.e., all its inputs will become zero except data input which becomes Z. The data output also becomes Z.
- The RAM supports both read and write operations. If an invalid address is specified for a write operation, the RAM does nothing. However if an invalid address is specified for a read operation the data read from the RAM will be Z..
- It does not support concurrent read and write operations



XYZ Corporation – Top Secret

2

Figure 1: Block Diagram of the Single-Port RAM (from the Design Specification)

Pin-Out Diagram and Pin Description

2. Pin Description

Pin-out information of the Single Port RAM:

Pin name	Direction	Width in bits	Functionality
clk	Input	1	Clock signal to the RAM
reset	Input	1	Active low reset signal, to initialize the RAM
write_enb	Input	1	Active high signal to enable write operation
read_enb	Input	1	Active high signal to enable the read operation
data_in	Input	8	Data input to the RAM
address	Input	5	Address to identify the memory location from 0 to 31
data_out	Output	8	Data output from the RAM

XYZ Corporation – Top Secret

3

Figure 2: Pin-Out Information of the Single-Port RAM (from the Design Specification)

4. Memory Operation Timing

Write Operation Timing

When reset is asserted, data_in is high-impedance and address is zero. When reset is released, write_enb=1 and read_enb=0, the data_in value is written into the corresponding memory address on the next positive clock edge.

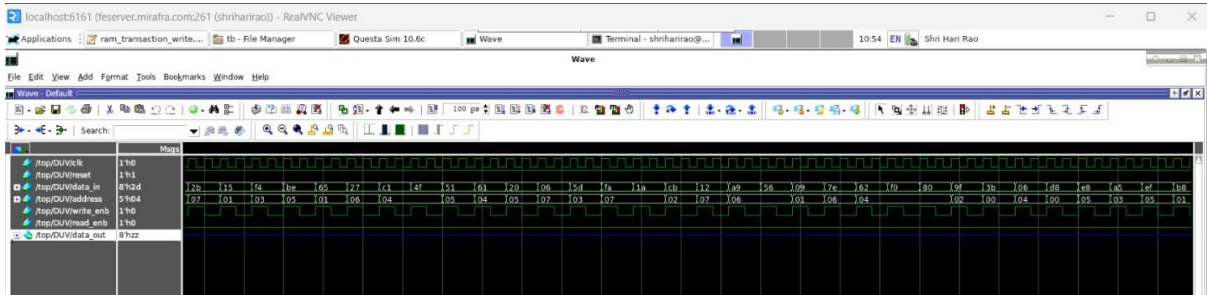


Figure 3: Memory Write Timing Diagram

Read Operation Timing

When reset is asserted, data_out is high-impedance and address is zero. When reset is released, write_enb=0 and read_enb=1, the data_out reflects the contents of the corresponding memory address.

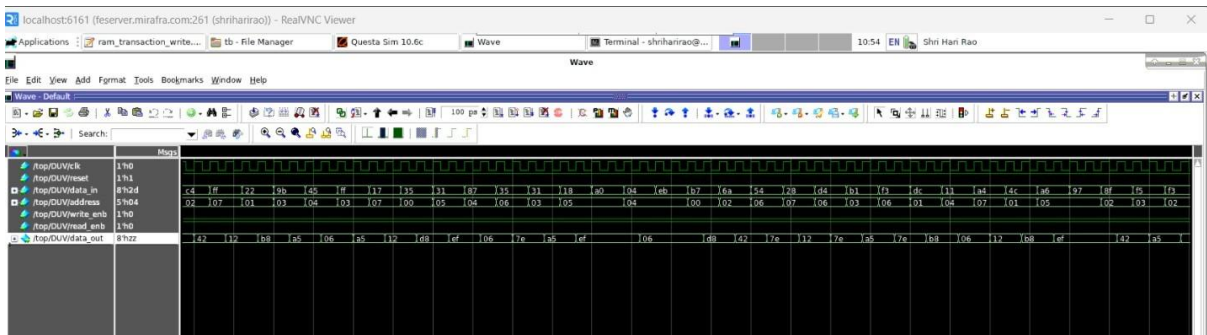


Figure 4: Memory Read Timing Diagram

5. Testbench Architecture

Testbench Block Diagram

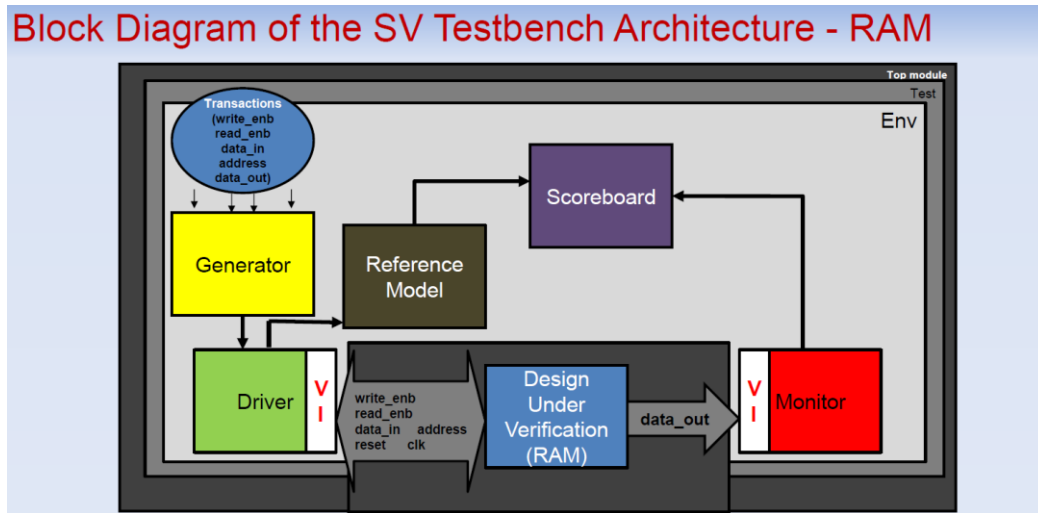


Figure 5: Block Diagram of the SV Testbench Architecture - RAM

Overall Structure

The testbench is a class-based SystemVerilog environment consisting of a generator, driver, monitor, reference model, and scoreboard, connected through mailboxes and coordinated by an environment class. A virtual interface with three modport views (DRV, MON, REF_SB) connects the testbench to the DUV, using clocking blocks to guarantee race-free signal timing.

Component Descriptions

ram_transaction - the stimulus data packet, carrying randomized `write_enb`, `read_enb`, `data_in`, and `address` fields, constrained to valid combinations.

ram_generator - randomizes transactions and forwards them to the driver via a mailbox.

ram_driver - drives randomized transactions onto the DUV's input pins through the DRV modport, and forwards the same transaction to the reference model.

ram_monitor - passively observes the DUV's `data_out` and `address` through the read-only MON modport and forwards captured results to the scoreboard.

ram_reference_model - an independent, behavioural golden model of the RAM's expected behaviour, used to compute the expected `data_out` for every read.

ram_scoreboard - compares the reference model's expected result against the monitor's observed result for every genuine read operation, and reports MATCH/MISMATCH counts.

ram_environment - constructs and wires together all components and runs them concurrently.

ram_test / test_write / test_read / test_regression - test classes that select which stimulus scenario the generator produces, by swapping the generator's blueprint transaction object.

6. Quality Assessment — Code Coverage

The testbench was simulated with code and functional coverage enabled in QuestaSim, across the full design hierarchy (interface, DUV, and every testbench class).

Overall Coverage Summary

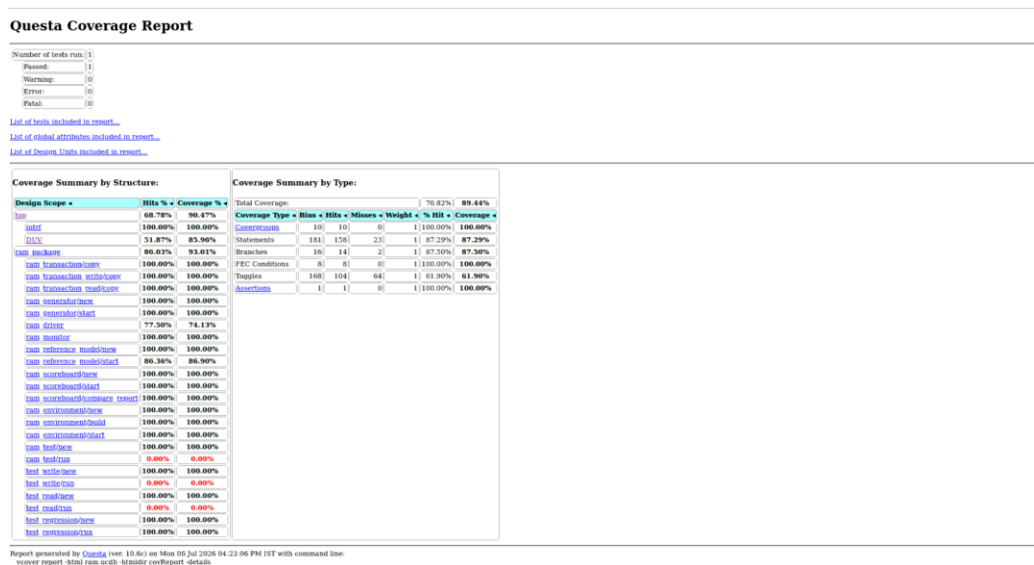


Figure 6: Questa Coverage Report — RAM Project

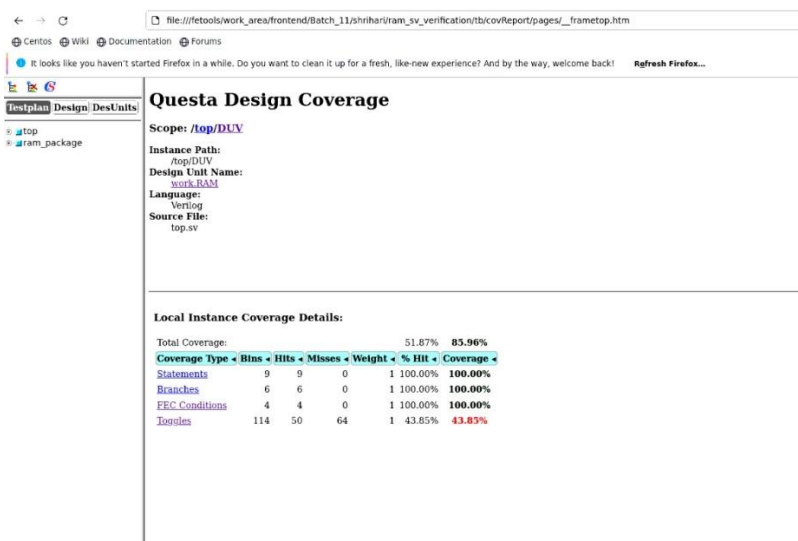


Figure 7: Questa code Coverage Report — RAM Project

Questa Toggle Coverage Report

Show All Show Covered Show Missing

Signal / Value	Hits						ExtMode	Status
	0L->1H	1H->0L	0L->Z	Z->1H	1H->Z	Z->0L		
address[0-4]	1	1	--	--	--	--	--	100.00%
clk	1	1	--	--	--	--	--	100.00%
data_in[0-7]	1	1	--	--	--	--	--	100.00%
data_out[7-0]	1	1	--	--	--	--	--	100.00%
i[31-0]	0	0	--	--	--	--	--	0.00%
read_enb	1	1	--	--	--	--	--	100.00%
reset	1	1	--	--	--	--	--	100.00%
write_enb	1	1	--	--	--	--	--	100.00%

Figure 7: Questa code Coverage Report — RAM Project

Key Observations

- Total coverage: 76.82% (Hits) / 89.44% (Coverage %) at the top level; the DUV itself reached 85.96% coverage.
- Statement coverage reached 87.29% (158 of 181 bins hit).
- Branch coverage reached 87.50% (14 of 16 bins hit).
- FEC Conditions reached 100% (8 of 8 bins hit).
- Functional covergroups reached 100% (10 of 10 bins hit) across the driver and monitor covergroups, including the write/read/data_in/address bins and the WRxRD cross with the illegal combination correctly excluded.
- Toggle coverage reached 61.90% (104 of 168 bins) - the largest remaining gap, concentrated in the driver, consistent with several reset- and timing-related branches not yet exercised by every stimulus path.
- Assertions reached 100% (1 of 1).
- test/run coverage for the standalone ram_test, test_write, and test_read classes remains at 0%, since only the consolidated test_regression class was executed in this run - each standalone class is functionally exercised indirectly through regression's internal segments, but its own run() task body has not been separately invoked.

7. Results

Simulation Results

The self-checking testbench was simulated using QuestaSim. The test_regression class was executed, generating 150 total transactions across three internal segments (base random, write-only, read-only; 50 transactions per segment). Of these, 79 were genuine read operations subject to scoreboard comparison.

Metric	Value
Total Transactions Generated	150
Genuine Reads Compared	79
MATCH	69
MISMATCH	10
Pass Rate (of compared reads)	87.34%

Documented Bug — Reset / Reference-Model Mismatch

Bug Description: When an address that was cleared by reset (but never explicitly written by any subsequent test operation) is read, the DUV and the reference model report different values. The reference model correctly reports 0, since the reference model's reset logic loops over all 32 memory locations and clears each one to 0. The DUV reports 'x' (uninitialized), since its reset branch only clears the single address that happens to be on the bus at that cycle, and clears it to 'z' rather than 0.

Root Cause: The DUV was not updated to satisfy the reference model's full-array reset behaviour. This is a design-side gap, not an testbench defect – the reference model's reset handling is correct and matches its own internal design intent.

Observed: All 10 mismatches in this run share the identical signature - duv model reports data_out='x', monitor reports data_out='0', for addresses that were reset but never subsequently written.

Expected: Reference model and DUV should agree on the memory state immediately following reset.

Status: Not yet fixed. Planned fix - update the design model's reset branch to loop over all DATA_DEPTH locations and clear each to 0, which will match exactly.

8. Conclusion

This project implemented a single-port RAM in synthesizable Verilog HDL, and verified it using a class-based SystemVerilog testbench following the generator-driver-monitor-scoreboard architecture with an independent behavioural reference model.

The testbench correctly verified write operations, read operations, read-after-write correctness, and the RAM's handling of simultaneous read/write requests, achieving 100% functional (covergroup) coverage and over 85% overall code coverage. One functional bug was identified: the DUV model's reset handling does not match the reference model's actual reset behaviour, producing 10 mismatches out of 79 compared reads in the current run. This is a design-side defect and has been root-caused precisely; the reference model itself behaves correctly.

9. Future Work

- Fix the documented design-model bug by updating its reset branch to clear all 32 memory locations to 0, matching the reference exactly, and re-run the regression to confirm 0 MISMATCH.
- Add a deliberate negative test (inject a known RTL bug, confirm the scoreboard reports MISMATCH, then revert) to prove the scoreboard is capable of detecting real functional faults.
- Add a mid-simulation reset pulse to exercise the driver's reset-handling branch while active, closing the remaining toggle and branch coverage gaps.
- Run the standalone ram_test, test_write, and test_read classes individually and merge their coverage databases with the regression run's, to close the remaining test/run code coverage gaps.
- Extend functional coverage to explicitly track read-after-write address correlation as a cross-coverage bin.
- Add assertion-based verification (SVA) for protocol-level checks such as write-then-read ordering and reset-to-idle timing.